AI Lab Record

Python and Prolog Programs

Index

1. Map Coloring (CSP)
2. Tic Tac Toe Game
3. Minimax Algorithm
4. Alpha-Beta Pruning
5. Decision Tree
6. Feed Forward Neural Network
7. Prolog: Sum of Integers from 1 to n
8. Prolog: Database of Name and Date of Birth
9. Prolog: Student, Teacher, Subject and Code
10. Prolog: Planets Database

1. Map Coloring (CSP)

Aim: To color the regions of a map using three colors so that no two neighbouring regions have the same color, using backtracking search for a Constraint Satisfaction Problem.

Program

```
# Map Coloring using Constraint Satisfaction Problem (backtracking)
variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
domain = ['Red', 'Green', 'Blue']
neighbors = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V': ['SA', 'NSW'],
    'T': []
}

def is_consistent(region, color, assignment):
    for n in neighbors[region]:
        if assignment.get(n) == color:
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment
    region = [v for v in variables if v not in assignment][0]
    for color in domain:
        if is_consistent(region, color, assignment):
            assignment[region] = color
            result = backtrack(assignment)
            if result:
                return result
            del assignment[region]
    return None

solution = backtrack({})
if solution:
    print("Map coloring solution:")
    for region in variables:
        print(region, "->", solution[region])
else:
    print("No solution exists")
```

Output

```
Map coloring solution:
WA -> Red
NT -> Green
SA -> Blue
Q -> Red
NSW -> Green
V -> Red
T -> Red
```

Result: The map was colored with no two adjacent regions sharing a color.

2. Tic Tac Toe Game

Aim: To write a two-player Tic Tac Toe game in Python.

Program

```
# Tic Tac Toe (two players)
board = [' '] * 9

def show():
    print()
    for i in range(0, 9, 3):
        print(' ' + ' | '.join(board[i:i+3]))
        if i < 6:
            print('---+---+---')
    print()

def winner():
    lines = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    for a, b, c in lines:
        if board[a] != ' ' and board[a] == board[b] == board[c]:
            return board[a]
    return None

player = 'X'
show()
while True:
    move = int(input("Player " + player + ", enter position (1-9): ")) - 1
    if move < 0 or move > 8 or board[move] != ' ':
        print("Invalid move, try again")
        continue
    board[move] = player
    show()
    w = winner()
    if w:
        print("Player", w, "wins!")
        break
    if ' ' not in board:
        print("It's a draw!")
        break
    player = 'O' if player == 'X' else 'X'
```

Output

```
   |   |
---+---+---
   |   |
---+---+---
   |   |

Player X, enter position (1-9):
 X |   |
---+---+---
   |   |
---+---+---
   |   |

Player O, enter position (1-9):
 X |   |
---+---+---
 O |   |
---+---+---
   |   |

Player X, enter position (1-9):
 X | X |
---+---+---
 O |   |
---+---+---
   |   |

Player O, enter position (1-9):
 X | X |
```

```
---+---+---
 O | O |
---+---+---
   |   |

Player X, enter position (1-9):
 X | X | X
---+---+---
 O | O |
---+---+---
   |   |

Player X wins!
```

Result: The game works and correctly detects a win or a draw. (Sample moves entered: 1, 4, 2, 5, 3.)

3. Minimax Algorithm

Aim: To implement the Minimax algorithm for a two-player game.

Program

```
# Minimax algorithm on a game tree (leaves hold utility values)
import math

tree = [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]   # depth 3, binary tree

def minimax(node, depth, is_max):
    if depth == 0:
        return node
    if is_max:
        best = -math.inf
        for child in node:
            best = max(best, minimax(child, depth - 1, False))
        return best
    else:
        best = math.inf
        for child in node:
            best = min(best, minimax(child, depth - 1, True))
        return best

print("Game tree:", tree)
print("Optimal value for MAX (root) =", minimax(tree, 3, True))
```

Output

```
Game tree: [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]
Optimal value for MAX (root) = 5
```

Result: The Minimax algorithm gave 5 as the best value MAX can get.

4. Alpha-Beta Pruning

Aim: To implement Alpha-Beta pruning, which gives the same answer as Minimax but skips branches that cannot change the result.

Program

```
# Alpha-Beta pruning
import math

tree = [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]
visited = 0

def alphabeta(node, depth, alpha, beta, is_max):
    global visited
    if depth == 0:
        visited += 1
        return node
    if is_max:
        best = -math.inf
        for child in node:
            best = max(best, alphabeta(child, depth - 1, alpha, beta, False))
            alpha = max(alpha, best)
            if beta <= alpha:
                print("Pruned at MAX node (alpha =", alpha, ", beta =", beta, ")")
                break
        return best
    else:
        best = math.inf
        for child in node:
            best = min(best, alphabeta(child, depth - 1, alpha, beta, True))
            beta = min(beta, best)
            if beta <= alpha:
                print("Pruned at MIN node (alpha =", alpha, ", beta =", beta, ")")
                break
        return best

print("Game tree:", tree)
value = alphabeta(tree, 3, -math.inf, math.inf, True)
print("Optimal value for MAX (root) =", value)
print("Leaf nodes evaluated:", visited, "out of 8")
```

Output

```
Game tree: [[[3, 5], [6, 9]], [[1, 2], [0, -1]]]
Pruned at MAX node (alpha = 6 , beta = 5 )
Pruned at MIN node (alpha = 5 , beta = 2 )
Optimal value for MAX (root) = 5
Leaf nodes evaluated: 5 out of 8
```

Result: Alpha-Beta pruning gave the same value (5) as Minimax but evaluated only 5 of the 8 leaf nodes.

5. Decision Tree

Aim: To build a decision tree with the ID3 algorithm (entropy and information gain) on the Play Tennis data set and predict a new sample.

Program

```
# Decision Tree (ID3 algorithm) on the Play Tennis data set
import math
from collections import Counter

attrs = ['Outlook', 'Temperature', 'Humidity', 'Wind']
data = [
 ['Sunny','Hot','High','Weak','No'],
 ['Sunny','Hot','High','Strong','No'],
 ['Overcast','Hot','High','Weak','Yes'],
 ['Rain','Mild','High','Weak','Yes'],
 ['Rain','Cool','Normal','Weak','Yes'],
 ['Rain','Cool','Normal','Strong','No'],
 ['Overcast','Cool','Normal','Strong','Yes'],
 ['Sunny','Mild','High','Weak','No'],
 ['Sunny','Cool','Normal','Weak','Yes'],
 ['Rain','Mild','Normal','Weak','Yes'],
 ['Sunny','Mild','Normal','Strong','Yes'],
 ['Overcast','Mild','High','Strong','Yes'],
 ['Overcast','Hot','Normal','Weak','Yes'],
 ['Rain','Mild','High','Strong','No'],
]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c/total) * math.log2(c/total) for c in counts.values())

def gain(rows, i):
    total = len(rows)
    rem = 0
    for v in set(r[i] for r in rows):
        subset = [r for r in rows if r[i] == v]
        rem += len(subset)/total * entropy(subset)
    return entropy(rows) - rem

def build(rows, available):
    labels = set(r[-1] for r in rows)
    if len(labels) == 1:
        return labels.pop()
    if not available:
        return Counter(r[-1] for r in rows).most_common(1)[0][0]
    best = max(available, key=lambda i: gain(rows, i))
    node = {attrs[best]: {}}
    rest = [a for a in available if a != best]
    for v in sorted(set(r[best] for r in rows)):
        node[attrs[best]][v] = build([r for r in rows if r[best] == v], rest)
    return node

def show(tree, indent=0):
    if not isinstance(tree, dict):
        print(' ' * indent + '-> ' + tree)
        return
    for a, branches in tree.items():
        for v, sub in branches.items():
            print(' ' * indent + a + ' = ' + v)
            show(sub, indent + 4)

def predict(tree, sample):
    while isinstance(tree, dict):
        a = list(tree.keys())[0]
        tree = tree[a][sample[a]]
    return tree

print("Information gain at root:")
for i, a in enumerate(attrs):
```

```
    print(" ", a, round(gain(data, i), 3))
tree = build(data, list(range(4)))
print("\nDecision tree:")
show(tree)
test = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
print("\nTest sample:", test)
print("Prediction (Play Tennis):", predict(tree, test))
```

Output

```
Information gain at root:
  Outlook 0.247
  Temperature 0.029
  Humidity 0.152
  Wind 0.048

Decision tree:
Outlook = Overcast
    -> Yes
Outlook = Rain
    Wind = Strong
        -> No
    Wind = Weak
        -> Yes
Outlook = Sunny
    Humidity = High
        -> No
    Humidity = Normal
        -> Yes

Test sample: {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
Prediction (Play Tennis): No
```

Result: Outlook has the highest information gain, so it is the root. The test sample was classified as No.

6. Feed Forward Neural Network

Aim: To implement a feed forward neural network with one hidden layer, trained with backpropagation, to learn the XOR function.

Program

```
# Feed Forward Neural Network (with backpropagation) learning XOR
import numpy as np

np.random.seed(1)
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

W1 = np.random.randn(2, 4)
b1 = np.zeros((1, 4))
W2 = np.random.randn(4, 1)
b2 = np.zeros((1, 1))
lr = 0.5

for epoch in range(10001):
    # forward pass
    h = sigmoid(X @ W1 + b1)
    out = sigmoid(h @ W2 + b2)
    error = y - out
    # backward pass
    d_out = error * out * (1 - out)
    d_h = (d_out @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d_out
    b2 += lr * d_out.sum(axis=0, keepdims=True)
    W1 += lr * X.T @ d_h
    b1 += lr * d_h.sum(axis=0, keepdims=True)
    if epoch % 2000 == 0:
        print("Epoch", epoch, "Loss:", round(float(np.mean(error ** 2)), 6))

print("\nInput  Predicted  Rounded")
for xi, o in zip(X, out):
    print(xi, " ", round(float(o[0]), 4), "  ", int(round(float(o[0]))))
```

Output

```
Epoch 0 Loss: 0.264921
Epoch 2000 Loss: 0.002302
Epoch 4000 Loss: 0.000807
Epoch 6000 Loss: 0.000471
Epoch 8000 Loss: 0.000328
Epoch 10000 Loss: 0.00025

Input  Predicted  Rounded
[0 0]   0.0085    0
[0 1]   0.9855    1
[1 0]   0.9837    1
[1 1]   0.0213    0
```

Result: The network learned XOR and the loss fell steadily during training.

7. Prolog: Sum of Integers from 1 to n

Aim: To write a Prolog program that finds the sum 1 + 2 + ... + n.

Program

```
% sum(N, S): S is the sum of integers from 1 to N
sum(0, 0).
sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is S1 + N.
```

Output

```
?- sum(5, S).
S = 15.

?- sum(10, S).
S = 55.

?- sum(100, S).
S = 5050.
```

Result: The program correctly gives the sum of the first n integers.

8. Prolog: Database of Name and Date of Birth

Aim: To create a Prolog database of names and dates of birth and query it.

Program

```
% dob(Name, Day/Month/Year)
dob(ravi,  12/5/2002).
dob(priya, 23/8/2001).
dob(kiran, 7/11/2002).
dob(anita, 30/1/2003).
```

Output

```
?- dob(priya, D).
D = 23/8/2001.

?- dob(N, _/_/2002).
N = ravi ;
N = kiran.

?- dob(anita, _/M/_).
M = 1.

?- dob(sam, D).
false.
```

Result: The database can be queried by name, year or month.

9. Prolog: Student, Teacher, Subject and Code

Aim: To create a Prolog database relating students, teachers, subjects and subject codes.

Program

```
% subject(Code, SubjectName)
subject(cs3491, ai).
subject(cs3451, dbms).
subject(cs3452, toc).

% teaches(Teacher, Code)
teaches(mr_rajan,   cs3491).
teaches(ms_lakshmi, cs3451).
teaches(mr_kumar,   cs3452).

% studies(Student, Code)
studies(arun,  cs3491).
studies(arun,  cs3451).
studies(bala,  cs3491).
studies(divya, cs3452).

% a teacher of a student is one who teaches a subject the student studies
teacher_of(Student, Teacher) :-
    studies(Student, Code),
    teaches(Teacher, Code).
```

Output

```
?- teacher_of(arun, T).
T = mr_rajan ;
T = ms_lakshmi.

?- studies(S, cs3491).
S = arun ;
S = bala.

?- subject(cs3451, N).
N = dbms.

?- teaches(mr_kumar, C), subject(C, N).
C = cs3452,
N = toc.
```

Result: The program finds the teachers of a student, the students of a subject and the subject name for a code.

10. Prolog: Planets Database

Aim: To create a Prolog database of planets and query it.

Program

```
% planet(Name, DistanceFromSunInAU, DiameterInKm)
planet(mercury, 0.39, 4879).
planet(venus,   0.72, 12104).
planet(earth,   1.0,  12742).
planet(mars,    1.52, 6779).
planet(jupiter, 5.2,  139820).

% inner planets are less than 2 AU from the sun
inner_planet(P) :- planet(P, D, _), D < 2.
```

Output

```
?- planet(earth, D, Dia).
D = 1.0,
Dia = 12742.

?- planet(P, _, Dia), Dia > 12000.
P = venus,
Dia = 12104 ;
P = earth,
Dia = 12742 ;
P = jupiter,
Dia = 139820.

?- inner_planet(P).
P = mercury ;
P = venus ;
P = earth ;
P = mars.
```

Result: The planets database can be queried by name, size and distance from the sun.